

Assignment 03 Final Report - NYC Taxi ETL & Analytics

This report summarizes ETL traceability from A2, row-count validation, analytical queries, business insights, visualizations, and optimization notes for the warehouse pipeline.

A2 Traceability Table

A2 Quality Issue	Transformation Applied	PySpark Function(s)
Duplicate rows	dropDuplicates()	dropDuplicates
Negative fare_amount	Distance-bin median imputation	when + percentile_approx + join
Zero/non-integer passenger_count	round and floor at min=1	round + greatest
Trip distance outliers	Cap at 99th percentile	percentile_approx + when
Tip greater than fare	Cap tip at cleaned fare	when
Datetime standardization	to_timestamp conversions	to_timestamp
Missing numeric values	Median imputation	coalesce + percentile_approx

Row Count Verification

Raw input rows: 2,964,624
fact_trips rows: 2,964,624
dim_datetime rows: 749
dim_payment rows: 5
dim_trip_category rows: 3

No unexplained row-loss was observed at fact level in this run. Dimension row-count differences are expected due to deduplication and dimensional modeling.

Q1 Hour-weekday volume and average fare - SQL Query

-- Business Q1: How do trip volume and average fare vary by hour and day of week?

```
SELECT
    d.pickup_weekday,
    d.pickup_hour,
    COUNT(*) AS trip_count,
    ROUND(AVG(f.fare_amount_clean), 2) AS avg_fare
FROM fact_trips f
JOIN dim_datetime d ON f.datetime_key = d.datetime_key
GROUP BY d.pickup_weekday, d.pickup_hour
ORDER BY
    CASE d.pickup_weekday
        WHEN 'Mon' THEN 1
        WHEN 'Tue' THEN 2
        WHEN 'Wed' THEN 3
        WHEN 'Thu' THEN 4
        WHEN 'Fri' THEN 5
        WHEN 'Sat' THEN 6
        WHEN 'Sun' THEN 7
        ELSE 8
    END,
    d.pickup_hour
```

Q1 Hour-weekday volume and average fare

pickup_weekday	pickup_hour	trip_count	avg_fare
Mon	0	11989	22.47
Mon	1	9832	21.59
Mon	2	7711	20.09
Mon	3	5999	21.21
Mon	4	4303	23.23
Mon	5	3828	25.86
Mon	6	7125	23.49
Mon	7	12645	19.91
Mon	8	16930	18.66
Mon	9	18139	19.08
Mon	10	19187	19.34
Mon	11	20725	18.25
Mon	12	23047	18.19
Mon	13	23734	19.36
Mon	14	26612	20.44
Mon	15	27326	19.69
Mon	16	26555	20.29
Mon	17	28113	18.77
Mon	18	28519	17.65
Mon	19	23532	19.16

Q1 Hour-weekday volume and average fare - Business Interpretation

This result quantifies how both demand (trip count) and pricing level (average fare) vary across weekday-hour slots. It helps identify when demand is high but average fares are low, and vice versa, which is useful for scheduling and pricing policies. Operations can align shift plans to high-volume slots while commercial teams target weaker slots with incentives.

Q2 Distance-fare correlation - SQL Query

-- Business Q2: What is the correlation between trip distance and fare amount?

```
SELECT
    ROUND(f.trip_distance_clean, 1) AS distance_bin,
    COUNT(*) AS trip_count,
    ROUND(AVG(f.fare_amount_clean), 2) AS avg_fare,
    ROUND(CORR(f.trip_distance_clean, f.fare_amount_clean), 4) AS distance_fare_corr
FROM fact_trips f
GROUP BY ROUND(f.trip_distance_clean, 1)
HAVING COUNT(*) >= 50
ORDER BY distance_bin
```

Q2 Distance-fare correlation

distance_bin	trip_count	avg_fare	distance_fare_corr
0.0	66701	25.59	-0.0059
0.1	6757	18.8	-0.1221
0.2	9986	8.04	-0.0909
0.3	24372	5.79	-0.0234
0.4	47789	5.85	0.026
0.5	74123	6.31	0.0348
0.6	97498	6.82	0.0567
0.7	113815	7.38	0.0448
0.8	122917	7.97	0.0517
0.9	125918	8.53	0.0569
1.0	125832	9.1	0.0546
1.1	121402	9.63	0.0474
1.2	116149	10.17	0.0496
1.3	110631	10.71	0.0457
1.4	104582	11.25	0.045
1.5	98865	11.76	0.0394
1.6	92121	12.24	0.0371
1.7	86002	12.72	0.042
1.8	79259	13.24	0.039
1.9	73446	13.72	0.0354

Q2 Distance-fare correlation - Business Interpretation

This query measures how strongly fare amount increases with trip distance and summarizes average fare at each distance bin. A strong positive correlation confirms pricing behavior is consistent with travel length. Deviations at specific bins can indicate fare policy edge-cases or unusual trip patterns worth investigation.

Q3 Payment preference by time and location - SQL Query

-- Business Q3: How does payment type preference change by time and location?

```
SELECT
    f.PULocationID,
    d.pickup_hour,
    p.payment_desc,
    COUNT(*) AS trip_count,
    ROUND(
        100.0 * COUNT(*) / SUM(COUNT(*) OVER (PARTITION BY f.PULocationID, d.pickup_hour),
        2
    ) AS payment_share_pct
FROM fact_trips f
JOIN dim_datetime d ON f.datetime_key = d.datetime_key
JOIN dim_payment p ON f.payment_type = p.payment_type
GROUP BY f.PULocationID, d.pickup_hour, p.payment_desc
HAVING COUNT(*) >= 100
ORDER BY trip_count DESC
LIMIT 200
```

Q3 Payment preference by time and location

PULocationID	pickup_hour	payment_desc	trip_count	payment_share_pct
161	18	Credit card	11509	83.06
161	17	Credit card	10825	82.58
161	19	Credit card	9787	83.77
236	15	Credit card	9776	81.22
237	14	Credit card	9691	81.73
237	15	Credit card	9279	81.52
161	16	Credit card	9132	81.6
237	17	Credit card	8537	82.97
162	18	Credit card	8524	84.69
237	16	Credit card	8496	82.65
236	17	Credit card	8447	82.07
237	18	Credit card	8438	82.17
236	14	Credit card	8409	80.52
237	13	Credit card	8402	80.3
132	16	Credit card	8355	74.82
236	12	Credit card	8347	81.31
161	15	Credit card	8323	80.63
236	16	Credit card	8312	82.78
237	12	Credit card	8238	80.14
161	14	Credit card	8076	80.06

Q3 Payment preference by time and location - Business Interpretation

This result shows payment mix by pickup location and hour, revealing behavioral differences by place and time. High card-share areas can prioritize digital payment reliability, while high-cash slots may need stronger cash-handling controls. The output supports localized customer experience and fraud-risk strategies.

Q4 Average tip by payment, distance, and vendor - SQL Query

-- Business Q4: How does average tip differ by payment type, distance, or vendor?

```
SELECT
    p.payment_desc,
    f.distance_bucket,
    f.VendorID,
    COUNT(*) AS trip_count,
    ROUND(AVG(f.tip_amount_clean), 2) AS avg_tip,
    ROUND(AVG(f.tip_amount_clean / NULLIF(f.fare_amount_clean, 0)) * 100, 2) AS avg_tip_pct_of_fare
FROM fact_trips f
JOIN dim_payment p ON f.payment_type = p.payment_type
GROUP BY p.payment_desc, f.distance_bucket, f.VendorID
HAVING COUNT(*) >= 100
ORDER BY avg_tip DESC
```

Q4 Average tip by payment, distance, and vendor

payment_desc	distance_bucket	VendorID	trip_count	avg_tip	avg_tip_pct_of_fare
Credit card	long	2	210333	11.58	21.17
Credit card	long	1	64829	8.85	16.88
Other	long	1	2332	5.43	10.89
Other	long	2	10839	5.3	10.94
Credit card	medium	2	529194	4.38	22.39
Credit card	medium	1	173895	4.05	21.47
Credit card	short	2	1014901	2.71	27.94
Credit card	short	1	325894	2.58	28.17
Other	medium	1	10028	2.07	11.3
Other	medium	2	45548	1.47	6.46
Other	short	2	35060	0.9	6.85
Other	short	1	36095	0.75	7.52
No charge	long	2	1002	0.11	0.17
Dispute	medium	2	9738	0.05	0.25
Dispute	long	2	5586	0.04	0.26
No charge	medium	2	1420	0.03	0.13
Dispute	short	2	27487	0.02	0.22
No charge	short	2	9055	0.01	0.13
Dispute	long	1	440	0.01	0.02
Cash	short	1	66629	0.0	0.0

Q4 Average tip by payment, distance, and vendor - Business Interpretation

This segmentation highlights where tipping behavior is strongest across payment type, trip distance class, and vendor. It helps compare vendor service outcomes and customer tipping propensity across contexts. The business can use it for driver coaching, incentive design, and payment-channel strategy.

Q5 Top pickup-dropoff location pairs - SQL Query

-- Business Q5: Which pickup/dropoff location pairs have highest volume and fare?

```
SELECT
    f.PULocationID,
    f.DOLocationID,
    COUNT(*) AS trip_count,
    ROUND(AVG(f.fare_amount_clean), 2) AS avg_fare,
    ROUND(SUM(f.total_revenue), 2) AS total_revenue,
    DENSE_RANK() OVER (ORDER BY COUNT(*) DESC, AVG(f.fare_amount_clean) DESC) AS pair_rank
FROM fact_trips f
GROUP BY f.PULocationID, f.DOLocationID
HAVING COUNT(*) >= 50
ORDER BY pair_rank
LIMIT 100
```

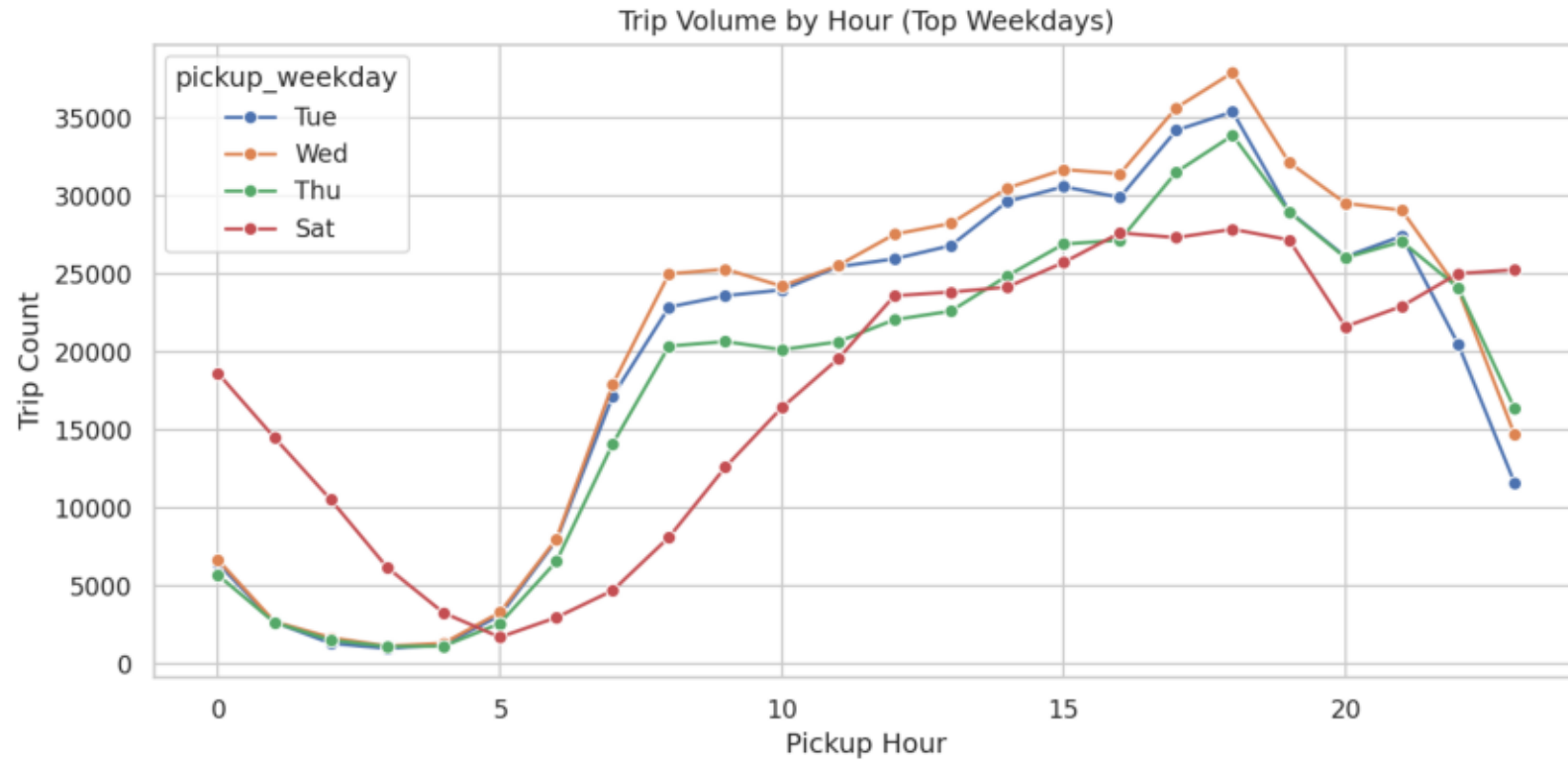

Q5 Top pickup-dropoff location pairs

PULocationID	DOLocationID	trip_count	avg_fare	total_revenue	pair_rank
237	236	21883	8.91	241483.51	1
236	237	19402	9.3	222381.04	2
236	236	15955	7.27	143620.3	3
237	237	14938	7.86	143576.19	4
161	237	10275	9.88	124696.53	5
142	239	8980	8.07	91000.95	6
237	161	8834	9.79	105381.94	7
161	236	8766	13.91	147659.52	8
239	142	8675	7.94	85841.85	9
239	238	8445	7.23	76881.93	10
141	236	7803	9.25	88351.45	11
237	162	7604	9.31	87234.85	12
264	264	7381	21.79	185424.43	13
132	132	7252	31.03	246050.6	14
236	161	7115	14.27	121459.44	15
186	230	7085	11.84	100413.25	16
142	238	7073	10.19	88794.95	17
238	239	7007	7.19	63194.05	18
236	141	6930	9.69	81942.13	19
236	239	6808	10.22	85604.91	20

Q5 Top pickup-dropoff location pairs - Business Interpretation

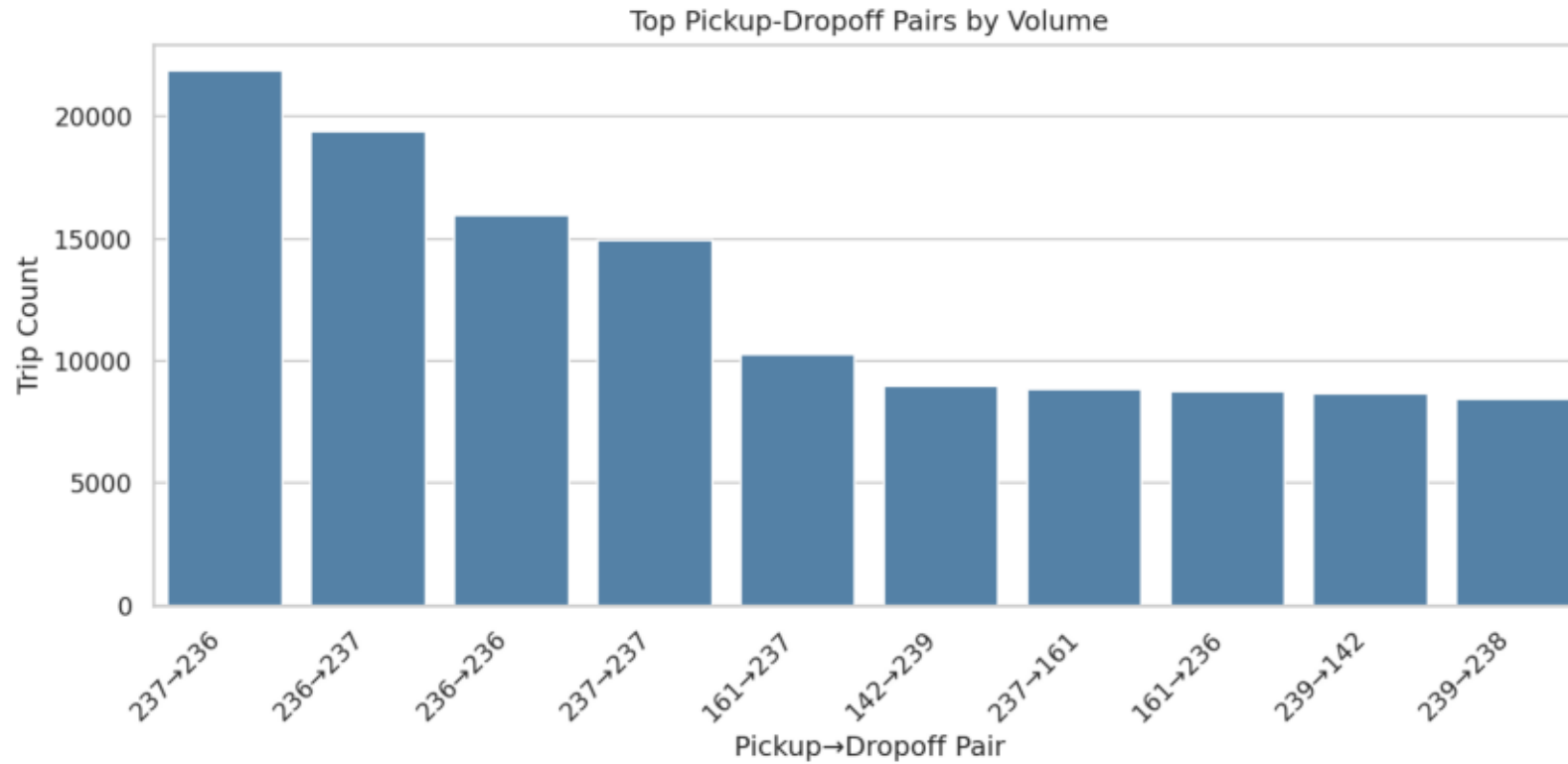
This query surfaces the OD corridors with the highest trip volume and associated fare behavior. These corridors are strategic routes for dispatch balancing, traffic monitoring, and service-level improvements. It also helps prioritize high-impact route partnerships and operational interventions.

Line Chart - Revenue Trend



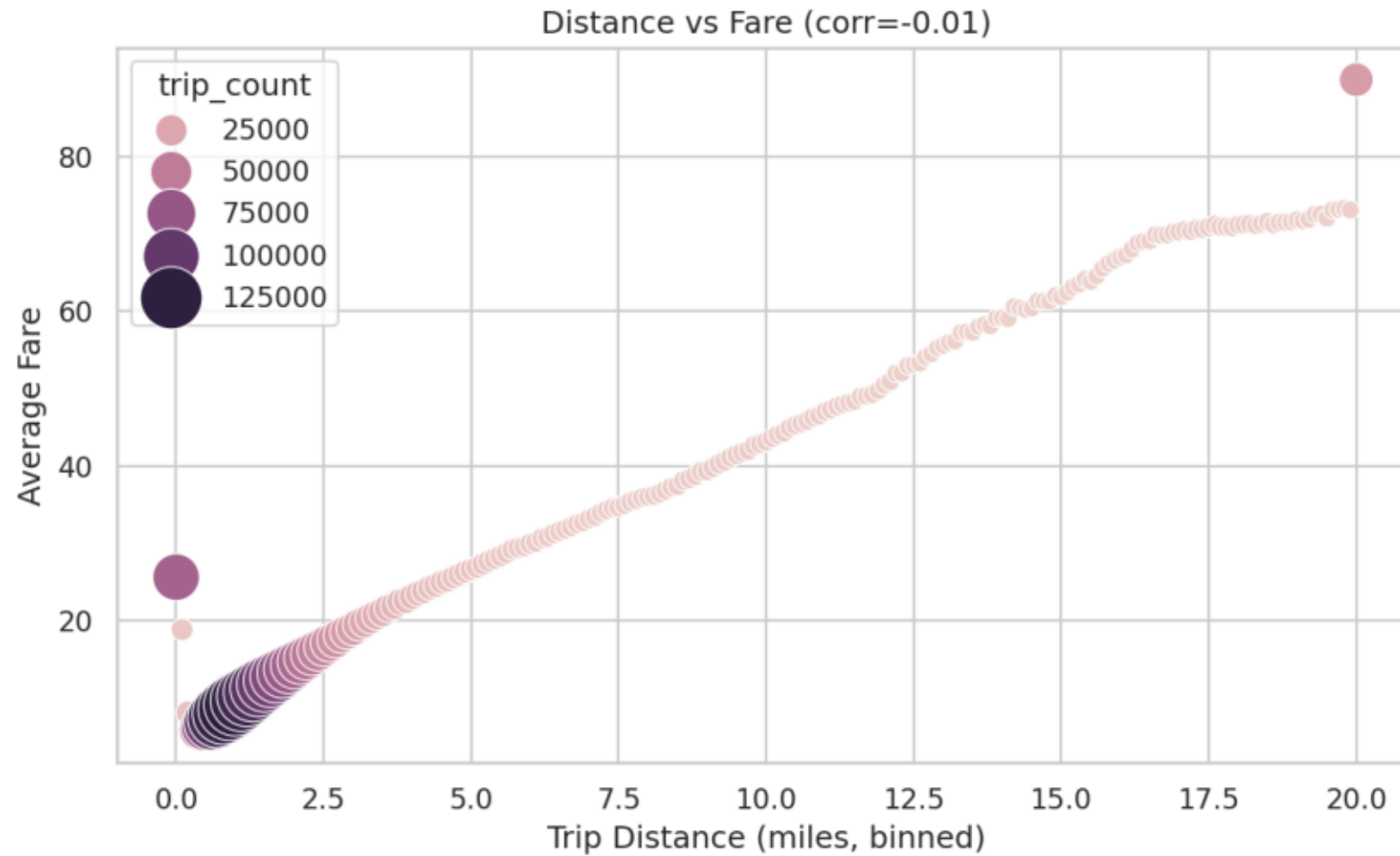
The line chart shows how trip volume changes by pickup hour for the busiest weekdays. It highlights demand peaks by time-of-day and supports staffing and fleet allocation decisions.

Bar Chart - Top Location Pairs



The bar chart identifies pickup-dropoff pairs with the highest trip volume. These corridors are priority candidates for dispatch optimization and service-quality monitoring.

Scatter Chart - Distance vs Fare



The scatter plot visualizes the relationship between trip distance and average fare. It also includes volume context by point size and supports validation of distance-based pricing behavior.

Dashboard (3+ subplots)

NYC Taxi Warehouse Dashboard



The dashboard combines volume, location-pair demand, payment preference, and tipping behavior in one view. Cross-reading these subplots helps explain both customer behavior and monetization patterns. This integrated view supports fast operational decision-making.

Pipeline Optimization Summary

Optimization techniques used:

- 1) Partitioning: fact and datetime tables are partitioned by pickup_year/pickup_month.
- 2) Caching: transformed DataFrame is cached before repeated dimension/fact derivations.
- 3) Broadcast joins: small dimensions are broadcast-joined to fact to reduce shuffle.
- 4) Query plan analysis: use Spark .explain(True) on complex window query during review.

Screenshot evidence of warehouse directory structure and data files in HDFS.